

Technical Report on Cinema Ticket Booking System

GROUP F MEMBERS

RUGABA BENEDICT-25/U/BIE/01417/PE

MUKINE JOEL-22/U/BIO/1688/PD

AUGUSTINE CEASER OLOYA-24/U/BIE/03944/PE

NAIWUMBWE ASSUMPTA-24/U/BIE/08557/PE

May 26, 2026

Contents

1	Introduction	3
2	System Build up and Data Structures	3
2.1	Movie Section	3
2.2	Booking Section	3
3	The Logic	3
3.1	Pricing Parameters	4
3.2	Surcharge and Group Discounts	4
4	Functional Matrix Breakdown	4
5	Conclusion & Future Work	4

1 Introduction

This report details the architectural design and technical implementation of a lightweight, terminal-based Cinema Ticket Booking System written in the C programming language. The application acts as a Point of Sale environment managing inventory, client reservations, dynamic tiered pricing structures, and financial auditing metrics.

2 System Build up and Data Structures

The system maintains data state utilizing two main *struct* definitions to control memory allocation arrays without running into pointer or memory leak issues.

2.1 Movie Section

The *Movie* structure manages the current physical seating state, inventory caps, and individual revenue tracking.

```
1 typedef struct {
2     int id;
3     char title[50];
4     int seats[20]; // 0 = Available, 1 = Booked
5     int tickets_sold;
6     float total_sales;
7 } Movie;
```

Listing 1: Movie Struct Definition

2.2 Booking Section

The *Booking* structure archives completed client transaction parameters. to be specific, the program obtains the name, the seat they want and other different parameters.

```
1 typedef struct {
2     int booking_id;
3     char customer_name[50];
4     int movie_id;
5     int seat_number;
6     char customer_type; // 'C' = Child, 'S' = Student, 'A' = Adult
7     int is_weekend;     // 1 = Yes, 0 = No
8     float final_price;
9 } Booking;
```

Listing 2: Booking Struct Definition

3 The Logic

Here we shall see the logic behind the source code.

3.1 Pricing Parameters

Let P_{base} represent the base cost depending on the customer age type T :

$$P_{base} = \begin{cases} \$5.00 & \text{if } T = 'C' \text{ (Child)} \\ \$8.00 & \text{if } T = 'S' \text{ (Student)} \\ \$12.00 & \text{if } T = 'A' \text{ (Adult)} \end{cases} \quad (1)$$

3.2 Surcharge and Group Discounts

Let W be the weekend modifier ($W = 0.20$ if weekend, 0 otherwise), and D be the group bulk discount modifier applied if total grouped tickets $N \geq 3$ ($D = 0.10$, 0 otherwise). The formula for final single-ticket evaluation P_{final} is modeled as follows:

$$P_{final} = (P_{base} + (P_{base} \times W)) \times (1 - D) \quad (2)$$

4 Functional Matrix Breakdown

The application exposes a structured pipeline for running administrative workflows.

Table 1: System Operations and Runtime Status

Function Call	Operational Scope	Implementation Status
<code>init_system()</code>	Bootstraps core catalogs	Complete
<code>book_tickets()</code>	Registers client data	Complete
<code>search_booking()</code>	Iterates via client matching tags	Complete
<code>update_booking()</code>	Swaps seat allocations safely	Complete
<code>cancel_booking()</code>	Purges records using ID masks	Complete
<code>display_available_seats()</code>	Renders matrix layout	Incomplete / Pending

5 Conclusion & Future Work

The system successfully implements robust processing for basic POS transaction flows. Future development phases will introduce:

- File I/O serialization to persist memory tables across session resets.
- Fixed size buffer overflow shields (%49[^\n] rules) for input capture safety.